

SIMATS ENGINEERING
(Saveetha School of Engineering)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1

Git & Docker Technical Assignment

Assigned Project: Online Food Delivery System

Tech Stack: React.js · Node.js/Express · MongoDB · Docker

Course Code: CSA1017 Course Title: Software Engineering

CO Assessed: CO3 Weightage: 20% Total Marks: 25

Student Name: R. Indhu

Register No: 192524332

1. Problem Analysis

1.1 Problem Statement

The assigned project is an Online Food Delivery System — a full-stack web application that allows customers to browse restaurant menus, place orders, make payments, and track deliveries, while restaurant administrators manage menu items and incoming orders through a dedicated dashboard.

1.2 Project Objectives

- Provide a responsive, easy-to-use interface for browsing menus and placing food orders.
- Implement secure user authentication and role-based access (customer vs. admin).
- Enable real-time order status tracking from placement to delivery.
- Containerize the application using Docker for consistent deployment across environments.
- Apply Git-based version control to support collaborative, traceable development.

1.3 Functional Requirements

ID	Requirement	Description
FR-1	User Registration/Login	Customers and admins can register and log in securely using JWT authentication.
FR-2	Menu Browsing	Customers can view categorized menu items with images, prices, and descriptions.
FR-3	Cart & Checkout	Customers can add items to cart, review order, and proceed to payment.
FR-4	Order Management	Admins can view, accept, and update the status of incoming orders.
FR-5	Order Tracking	Customers can track live status of their order (Placed → Preparing → Out for Delivery → Delivered).

1.4 Expected Outcomes

A fully functional, containerized full-stack application with a documented Git history demonstrating collaborative development practices, and a working Docker deployment that can be reproduced on any machine with a single command.

2. Project Structure Design

The repository follows a modular, service-oriented folder structure that separates the frontend and backend concerns, keeps configuration files at the root for easy discovery, and isolates documentation and Docker-related files.

```

online-food-delivery/
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   └── App.js
│   ├── package.json
│   └── Dockerfile
├── backend/
│   ├── src/
│   │   ├── controllers/
│   │   ├── models/
│   │   ├── routes/
│   │   └── server.js
│   ├── package.json
│   └── Dockerfile
├── docs/
│   └── architecture-diagram.png
├── .gitignore
├── README.md
└── docker-compose.yml

```

Figure 1: Repository folder structure

2.1 Purpose of Key Folders and Files

Folder / File	Purpose
frontend/	Contains the React.js single-page application (UI components, pages, routing).
backend/	Contains the Node.js/Express REST API (controllers, models, routes, server entry point).
backend/src/models/	Mongoose schemas defining User, MenuItem, and Order data structures.
backend/src/controllers/	Business logic handling requests for authentication, orders, and menu operations.
docs/	Architecture diagrams and supporting documentation.
.gitignore	Excludes node_modules, .env, and build artifacts from version control.
README.md	Setup instructions, environment variables, and usage guide.
docker-compose.yml	Orchestrates frontend, backend, and database containers together.

3. Git Repository Initialization

The Git repository was initialized locally and then connected to a remote hosting service (GitHub) to enable collaboration and backup.

3.1 Initialization Steps

```
$ git init
$ git add .
$ git commit -m "Initial project scaffolding"
$ git branch -M main
$ git remote add origin https://github.com/team/online-food-delivery.git
$ git push -u origin main
```

3.2 Purpose of Essential Git Files

File	Purpose
.git/	Hidden directory storing the entire version history, object database, and references.
.gitignore	Specifies files/folders (node_modules, .env, build/) that Git should not track.
README.md	Documents project setup, purpose, and contribution guidelines for collaborators.

4. Git Branching Strategy

A Gitflow-inspired branching model was adopted, using dedicated branches for ongoing development, isolated feature work, release stabilization, and urgent production fixes.

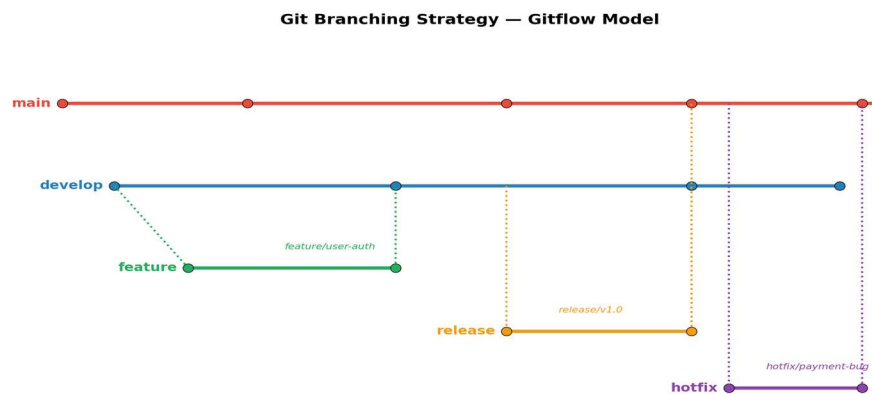


Figure 2: Git branching strategy (Gitflow model)

4.1 Branch Roles

Branch	Purpose
main	Always reflects the stable, production-ready state of the application.

Branch	Purpose
develop	Integration branch where completed features are merged before release.
feature/*	Short-lived branches for building individual features (e.g., feature/user-auth).
release/*	Used to stabilize and test a version before merging into main.
hotfix/*	Created from main to urgently patch production issues.

4.2 Justification

This model suits the project because it cleanly separates in-progress work (develop, feature branches) from stable, deployable code (main), which matters for a food-delivery application where production reliability (order and payment flows) is critical. It also gives the team a clear path for emergency fixes via hotfix branches without disturbing ongoing feature development.

5. Version Control Workflow

The Git workflow below demonstrates branch creation, meaningful commits, merging into the integration branch, and synchronization with the remote repository.

```

bash — git workflow

$ git checkout -b feature/user-auth develop
Switched to a new branch 'feature/user-auth'

$ git add .
$ git commit -m "Add JWT-based authentication module"
[feature/user-auth 8e1f2a9] Add JWT-based authentication module
4 files changed, 182 insertions(+)

$ git checkout develop
$ git merge feature/user-auth
Merge made by the 'recursive' strategy.

$ git push origin develop

```

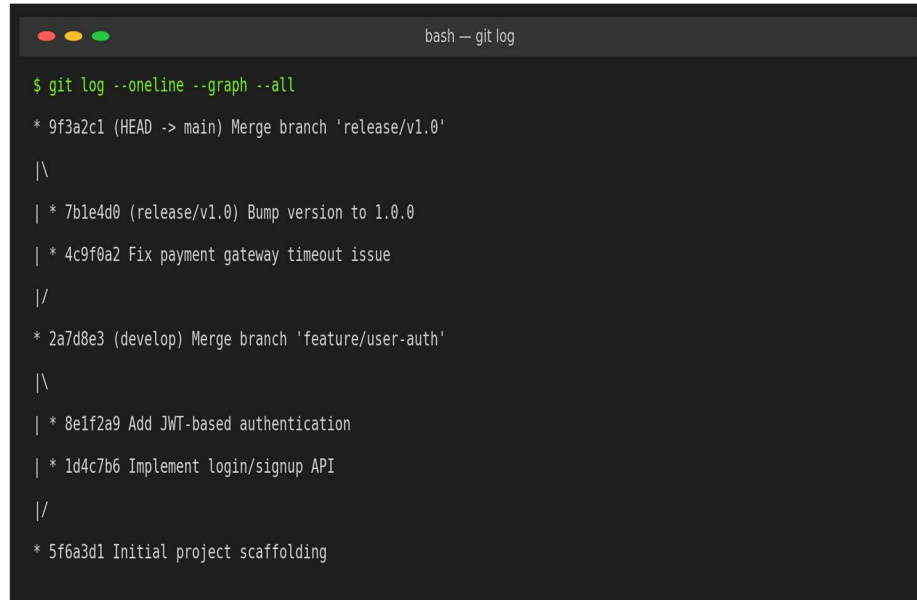
Figure 3: Git workflow — branch creation, commit, and merge

5.1 Explanation of Operations

Operation	Explanation
git checkout -b	Creates and switches to a new feature branch off develop.
git add / commit	Stages and records a meaningful, atomic snapshot of changes.
git merge	Integrates completed feature work back into develop.
git push	Synchronizes local commits with the shared remote repository.

Conflict resolution: During the merge of feature/user-auth, a conflict occurred in backend/src/routes/index.js because two branches modified the same import block. It was resolved by manually reviewing both changes, retaining both route registrations, and committing the merge with git commit after running git status to confirm a clean working tree.

6. Commit History Analysis



```
bash — git log

$ git log --oneline --graph --all
* 9f3a2c1 (HEAD -> main) Merge branch 'release/v1.0'
|\
| * 7b1e4d0 (release/v1.0) Bump version to 1.0.0
| * 4c9f0a2 Fix payment gateway timeout issue
|/
* 2a7d8e3 (develop) Merge branch 'feature/user-auth'
|\
| * 8elf2a9 Add JWT-based authentication
| * 1d4c7b6 Implement login/signup API
|/
* 5f6a3d1 Initial project scaffolding
```

Figure 4: Commit history (git log --oneline --graph --all)

6.1 Commit Message Practices

- Each commit uses a concise, imperative summary line (e.g., "Add JWT-based authentication").
- Commits are atomic — each one represents a single logical change, making history easy to review and revert if needed.
- Merge commits clearly indicate which branch was integrated, preserving traceability.
- This discipline simplifies debugging (via git bisect), code review, and onboarding new contributors.

7. Docker Environment Design

7.1 Why Docker is Suitable

Docker ensures the application runs identically across development, testing, and production machines by packaging each service with its own dependencies, eliminating "it works on my machine" issues. It also allows the frontend, backend, and database to be scaled and deployed independently.

7.2 Components Requiring Containerization

Component	Reason for Containerization
React Frontend	Isolates Node/npm build tooling and serves the compiled static bundle consistently.

Component	Reason for Containerization
Node.js/Express Backend	Packages the API runtime, dependencies, and environment configuration together.
MongoDB	Provides a disposable, reproducible database instance with persistent volume storage.
Nginx (optional)	Acts as a reverse proxy routing traffic between frontend and backend containers.

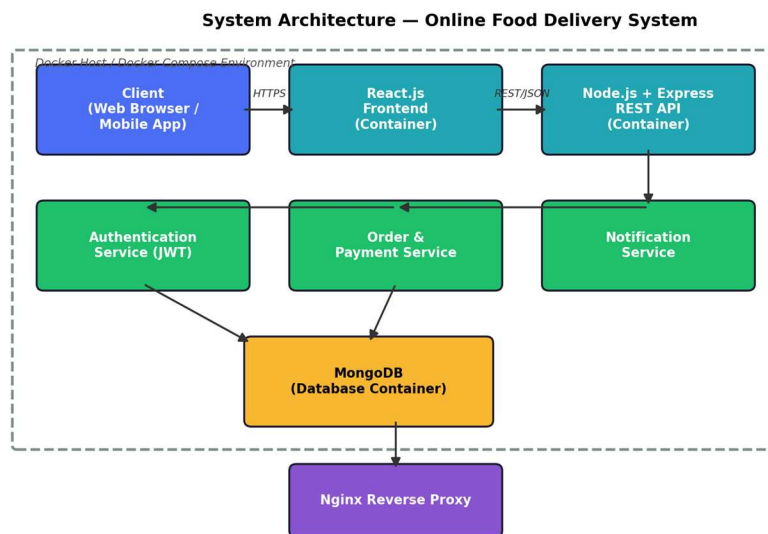


Figure 5: Containerized system architecture

8. Dockerfile Development

Separate Dockerfiles were created for the frontend and backend to keep each image minimal and purpose-built.

8.1 Backend Dockerfile

```

# backend/Dockerfile
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --production
COPY . .
EXPOSE 5000
CMD ["node", "src/server.js"]

```

8.2 Frontend Dockerfile

```

# frontend/Dockerfile
FROM node:18-alpine AS build

```

```

WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

FROM nginx:stable-alpine
COPY --from=build /app/build /usr/share/nginx/html
EXPOSE 3000

```

8.3 Instruction Explanation

Instruction	Purpose
FROM	Defines the base image (Node.js or Nginx) the container builds upon.
WORKDIR	Sets the working directory inside the container for subsequent commands.
COPY	Copies source files/dependencies from host into the image.
RUN	Executes build-time commands such as installing dependencies or building the app.
EXPOSE	Documents the port the container listens on.
CMD	Specifies the default command executed when the container starts.

9. Docker Compose Design

Docker Compose orchestrates all three services — frontend, backend, and database — defining how they network together, persist data, and receive configuration.

```

# docker-compose.yml
version: "3.9"
services:
  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
    depends_on:
      - backend

  backend:
    build: ./backend
    ports:
      - "5000:5000"
    environment:

```



```
- MONGO_URI=mongodb://mongodb:27017/fooddelivery
- JWT_SECRET=${JWT_SECRET}
depends_on:
- mongodb
```

```
mongodb:
  image: mongo:6.0
  ports:
  - "27017:27017"
  volumes:
  - mongo-data:/data/db
```

```
volumes:
  mongo-data:
```

9.1 Configuration Explanation

Element	Explanation
services	Defines the three containers: frontend, backend, and mongodb.
build	Builds an image from the corresponding Dockerfile rather than pulling one.
ports	Maps container ports to host ports for external access.
environment	Injects runtime configuration such as the database URI and JWT secret.
depends_on	Ensures backend starts after mongodb, and frontend after backend.
volumes	Persists MongoDB data outside the container lifecycle so it survives restarts.

Docker Build & Deployment Workflow

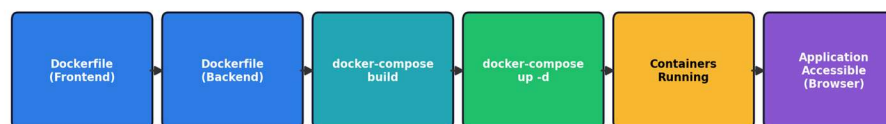


Figure 6: Docker build and deployment workflow

10. Container Deployment and Testing

The application was deployed by running `docker-compose build` followed by `docker-compose up -d`, then verified by checking container status and accessing the running application in a browser.

```
bash — docker-compose build

$ docker-compose build

[+] Building 24.6s (18/18) FINISHED
=> [backend] load build definition      0.1s
=> [backend] load .dockerignore        0.0s
=> [backend] npm install               12.3s
=> [backend] COPY . .                  0.4s
=> [frontend] npm run build            9.8s
=> exporting to image                  1.2s

Successfully built backend:latest
Successfully built frontend:latest
```

Figure 7: docker-compose build output

```
bash — docker ps

$ docker-compose up -d

Creating network "foodapp_default"
Creating foodapp_mongodb_1 ... done
Creating foodapp_backend_1 ... done
Creating foodapp_frontend_1 ... done

$ docker ps

CONTAINER ID   IMAGE             STATUS              PORTS
a1b2c3d4e5f6   foodapp_frontend  Up 2 minutes       0.0.0.0:3000->3000
b2c3d4e5f6a1   foodapp_backend   Up 2 minutes       0.0.0.0:5000->5000
c3d4e5f6a1b2   mongo:6.0         Up 2 minutes       0.0.0.0:27017->27017
```

Figure 8: Containers running (docker ps)

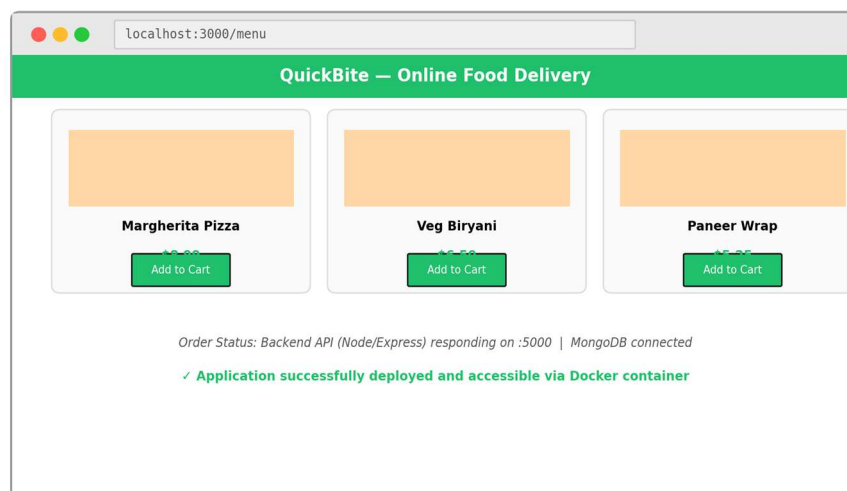


Figure 9: Application accessible in the browser at localhost:3000

10.1 Testing Procedure

- Verified all three containers show "Up" status via docker ps.
- Accessed the frontend at http://localhost:3000 and confirmed the menu page loads correctly.
- Tested API endpoints (e.g., /api/menu, /api/orders) using Postman to confirm correct responses.
- Confirmed MongoDB persistence by restarting containers and verifying order data remained intact.

11. Benefits of Git and Docker

Area	Git	Docker
Collaboration	Multiple developers work on isolated branches without conflicts.	Every developer runs an identical environment, avoiding setup mismatches.
Version Management	Full history of changes with the ability to revert or compare versions.	Image tags allow rolling back to a previous working build.
Portability	Repository can be cloned and run on any machine.	Containers run identically across dev, test, and production.
Deployment	Enables CI/CD pipelines triggered by pushes/merges.	Simplifies deployment to any Docker-compatible host or cloud service.
Scalability	Branching supports parallel feature development at scale.	Services can be scaled independently (e.g., multiple backend replicas).
Maintainability	Clear commit history aids debugging and code review.	Isolated services are easier to update without affecting others.

12. Challenges and Improvements

12.1 Challenges Encountered

- Merge conflicts arose when multiple feature branches modified shared route files simultaneously.
- Initial Docker images were unnecessarily large due to including development dependencies in the production image.
- Environment variable management across local and containerized environments required careful handling to avoid leaking secrets.

12.2 Suggested Improvements

- Adopt multi-stage builds more extensively to further reduce final image size.
- Introduce a CI/CD pipeline (GitHub Actions) to automatically run tests and build Docker images on every push.
- Enforce branch protection rules and mandatory pull-request reviews on main and develop.

- Add a `.env.example` file and use Docker secrets for more secure credential management in production.

12.3 Future Enhancements

- Integrate container orchestration (Kubernetes) for auto-scaling during high order volumes.
- Add automated integration tests that run inside containers before deployment.
- Implement real-time order tracking using WebSockets for a more responsive user experience.